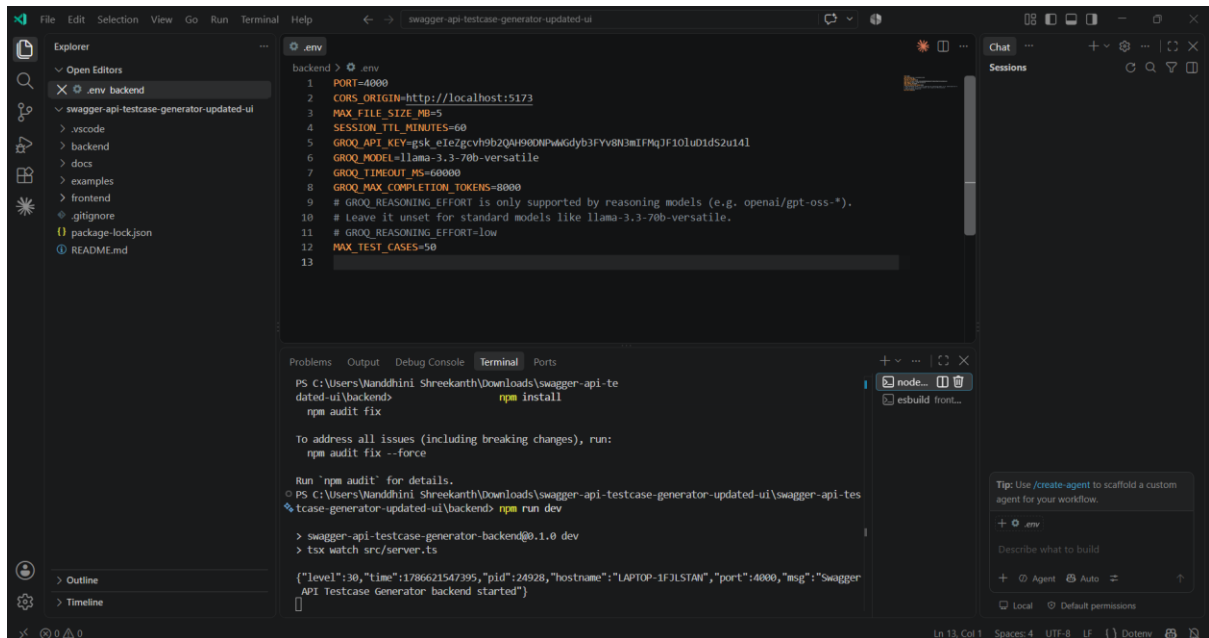


1. Generate Reliable API Test Cases from Swagger

Restful-booker-openapi.yaml

Run the backend with npm run dev



The screenshot shows the VS Code interface with the `.env` file open in the editor. The file contains the following configuration:

```
1 PORT=4000
2 CORS_ORIGIN=http://localhost:5173
3 MAX_FILE_SIZE_MB=5
4 SESSION_TTL_MINUTES=60
5 GROQ_API_KEY=gsk_eIeZgcVh9b2QAH900NPwGdyb3FYv8N3mTFMqJF10lUd1dS2u14l
6 GROQ_MODEL=llama-3.3-70b-versatile
7 GROQ_TIMEOUT_MS=60000
8 GROQ_MAX_COMPLETION_TOKENS=8000
9 # GROQ_REASONING_EFFORT is only supported by reasoning models (e.g. openai/gpt-oss-*).
10 # Leave it unset for standard models like llama-3.3-70b-versatile.
11 # GROQ_REASONING_EFFORT=low
12 MAX_TEST_CASES=50
13
```

The terminal shows the command `npm install` and `npm run dev` being executed. The output indicates that the backend is running successfully on port 4000.

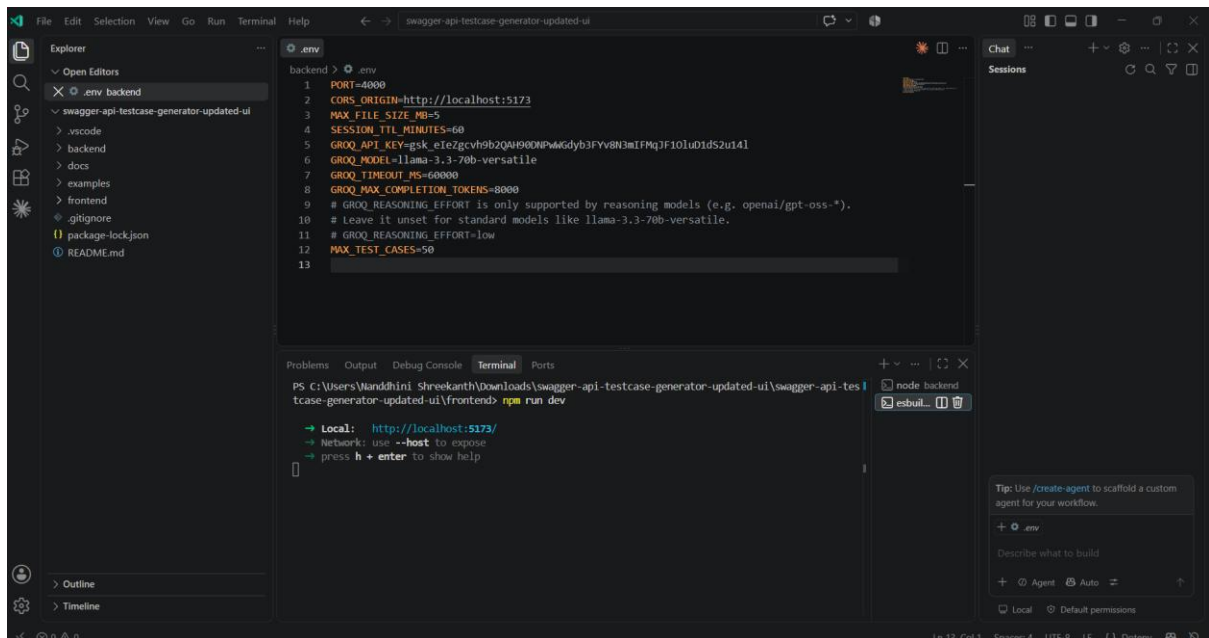
```
PS C:\Users\Ianddhini Shreekanth\Downloads\swagger-api-testcase-generator-updated-ui> npm install
npm audit fix

To address all issues (including breaking changes), run:
npm audit fix --force

Run 'npm audit' for details.
PS C:\Users\Ianddhini Shreekanth\Downloads\swagger-api-testcase-generator-updated-ui\backend> npm run dev
> swagger-api-testcase-generator-backend@0.1.0 dev
> tsx watch src/server.ts

{"level":30,"time":1786621547395,"pid":24928,"hostname":"LAPTOP-1FJLSTAN","port":4000,"msg":"Swagger
API Testcase Generator backend started"}
```

Run the front end with npm run dev

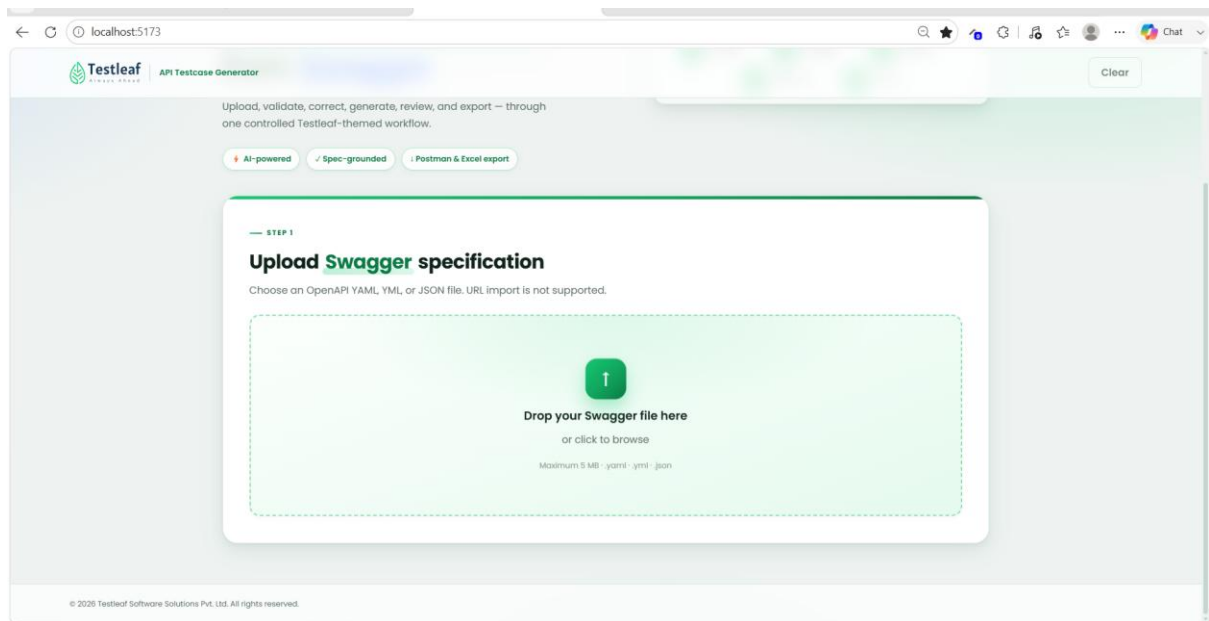


The screenshot shows the VS Code interface with the `.env` file open in the editor. The file contains the same configuration as the previous screenshot.

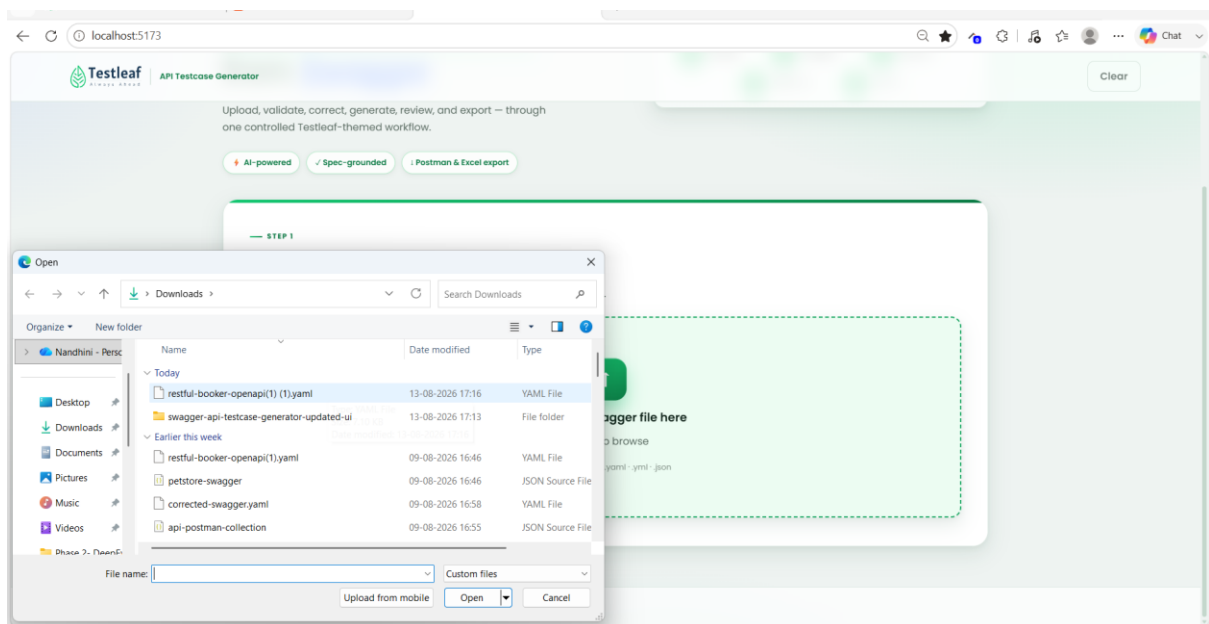
The terminal shows the command `npm run dev` being executed. The output indicates that the frontend is running successfully on port 5173.

```
PS C:\Users\Ianddhini Shreekanth\Downloads\swagger-api-testcase-generator-updated-ui\swagger-api-testcase-generator-updated-ui\frontend> npm run dev
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

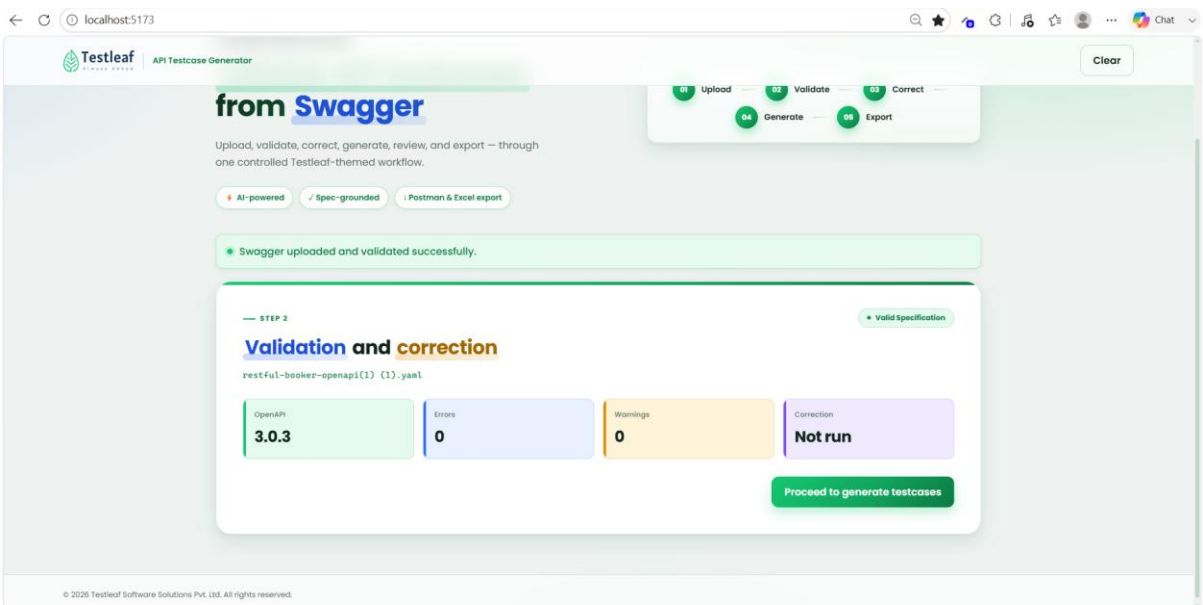
UI



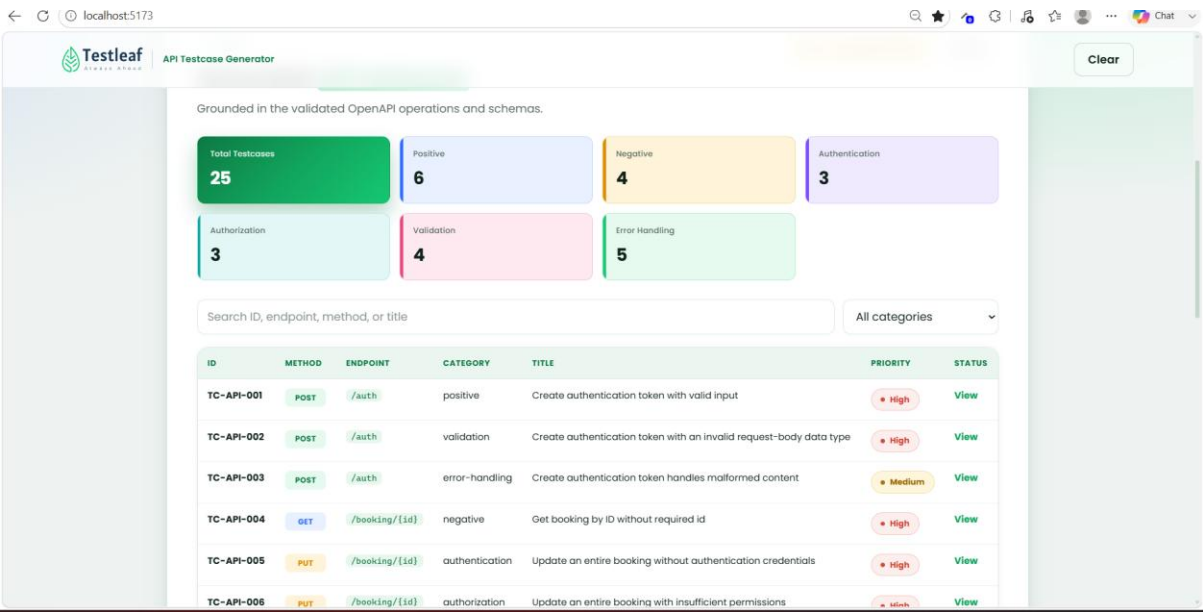
Uploading restful booker.yaml file



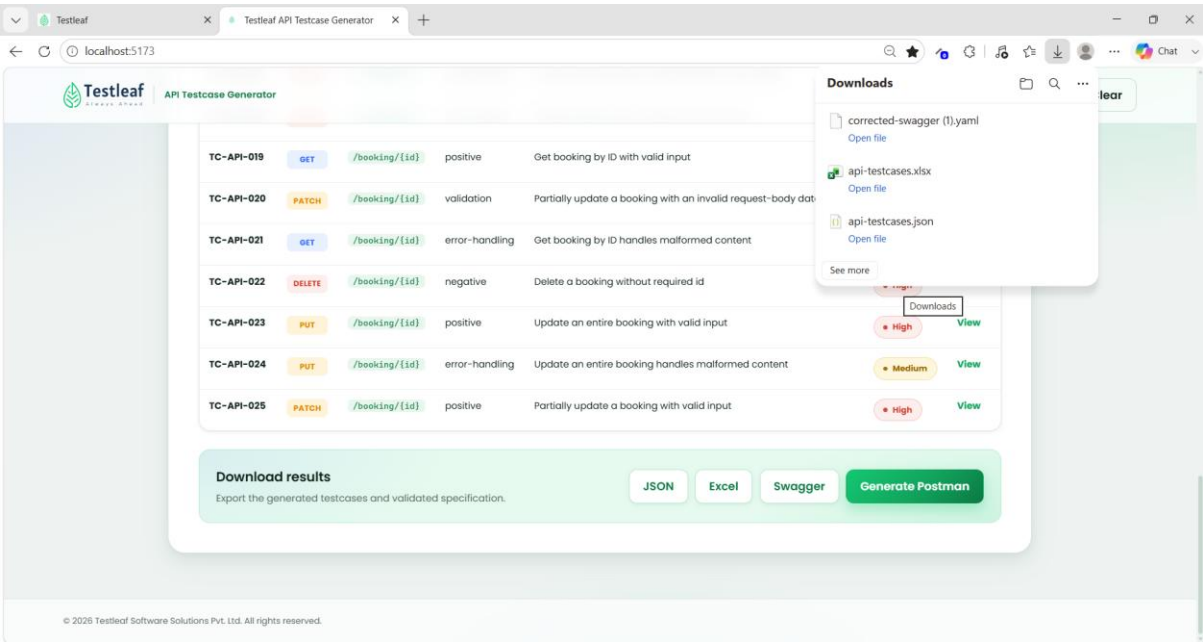
Uploaded successfully with no errors



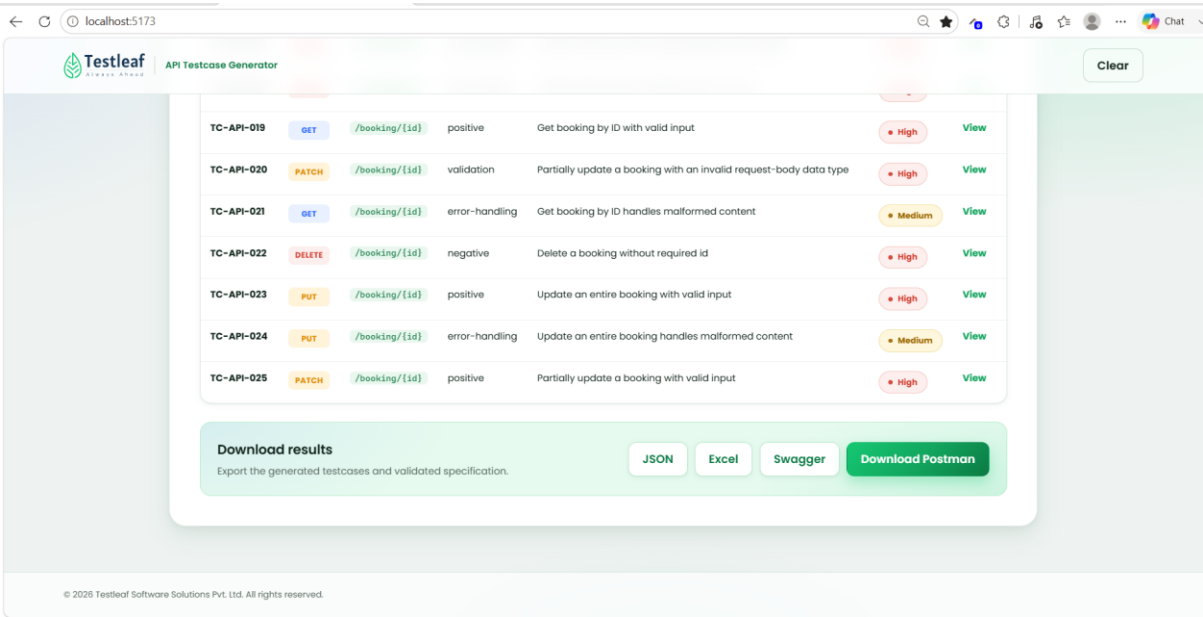
Click Proceed to generate testcases



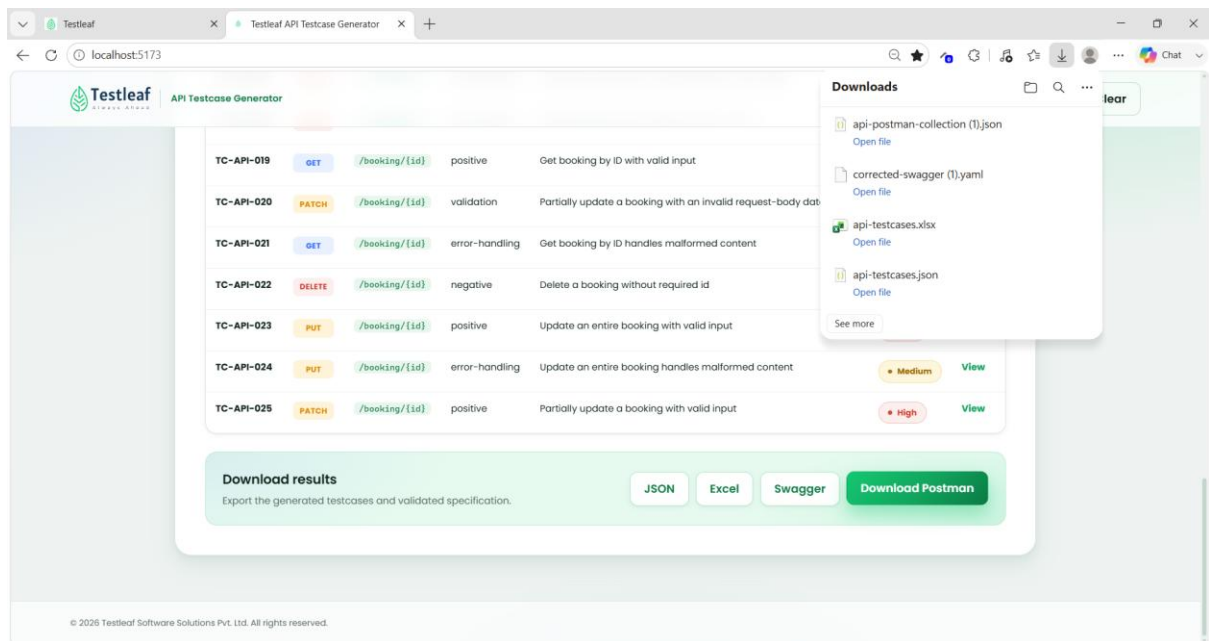
Export the generated test cases through Json/Excel/Swagger



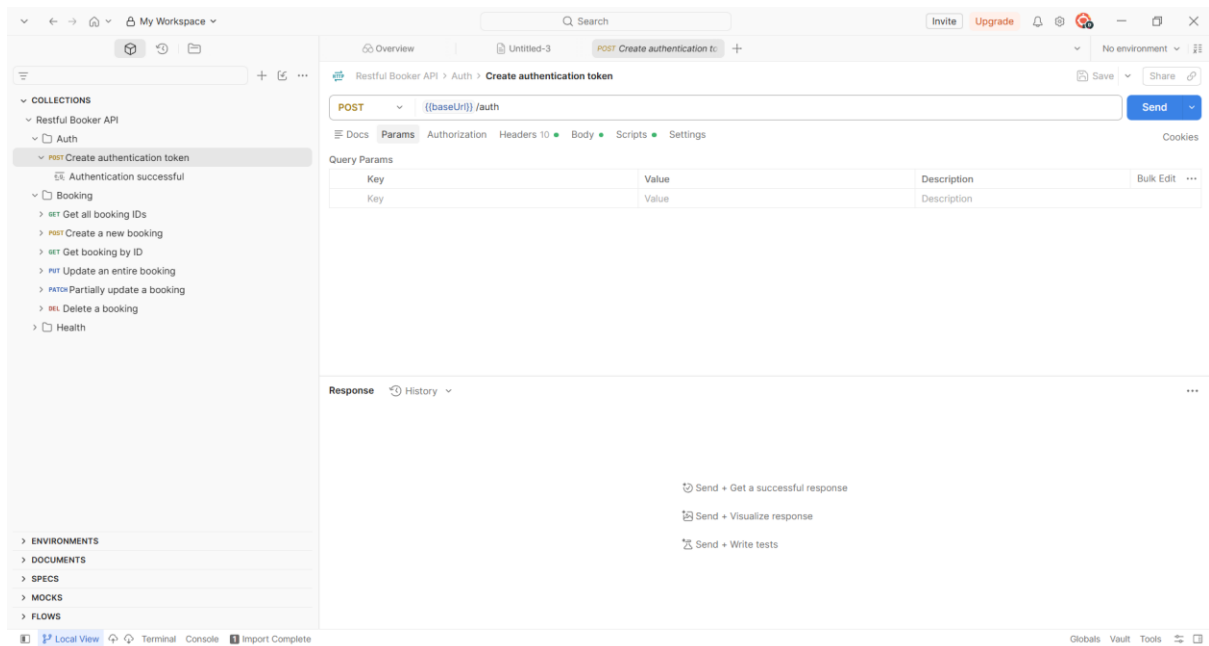
Generate Postman Collection is clicked, it will ask for download postman



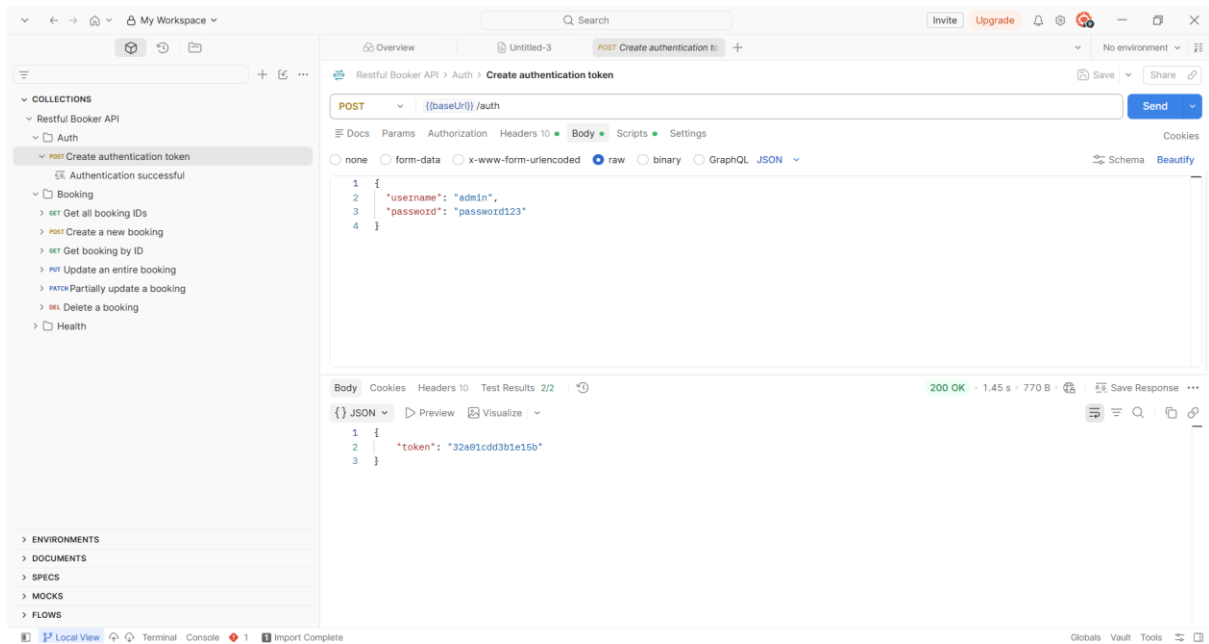
Click Download Postman



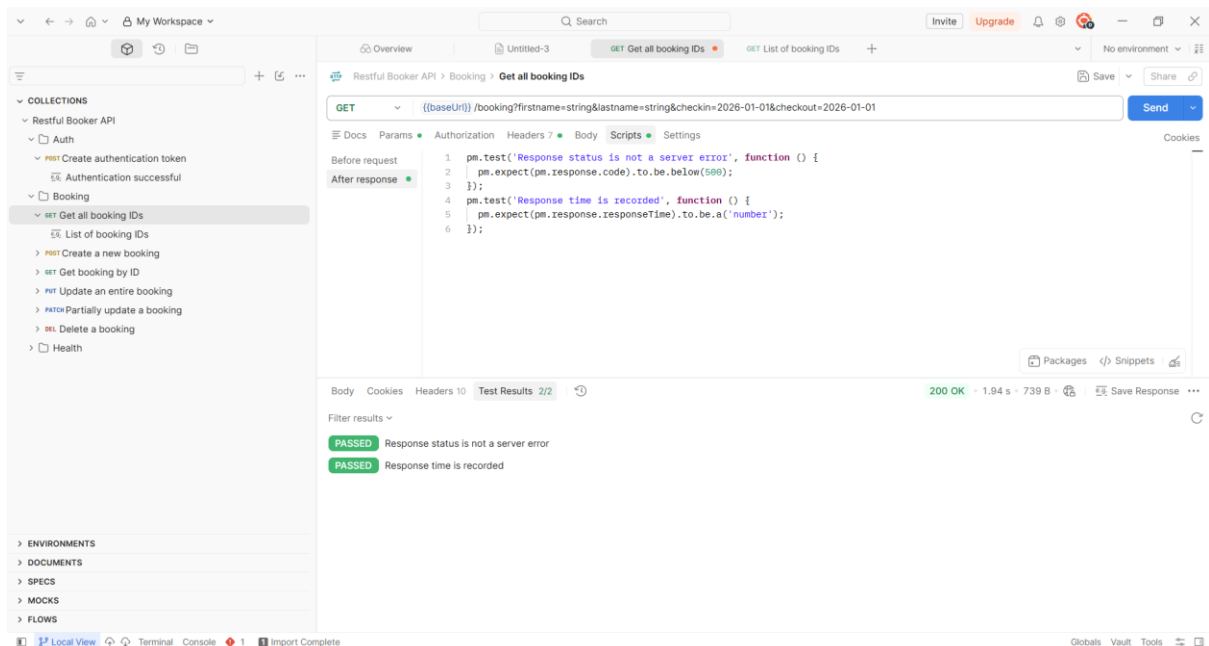
Open the downloaded Postman file in Postman



Verified Create Authentication token



Verified Get All Booking IDs



Create a new booking ID

The screenshot shows the Postman interface with a REST client collection named "Restful Booker API". The "Booking" folder is expanded, and the "Create a new booking" endpoint is selected. The request is a POST to `{{baseUri}}/booking` with a JSON body:

```
1 {
2   "firstname": "Jim",
3   "lastname": "Brown",
4   "totalprice": 111,
5   "depositpaid": true,
6   "bookingdates": {
7     "checkin": "2026-08-10",
8     "checkout": "2026-08-15"
9   },
10  "additionalneeds": "Breakfast"
11 }
```

The response is a 200 OK status with a JSON body:

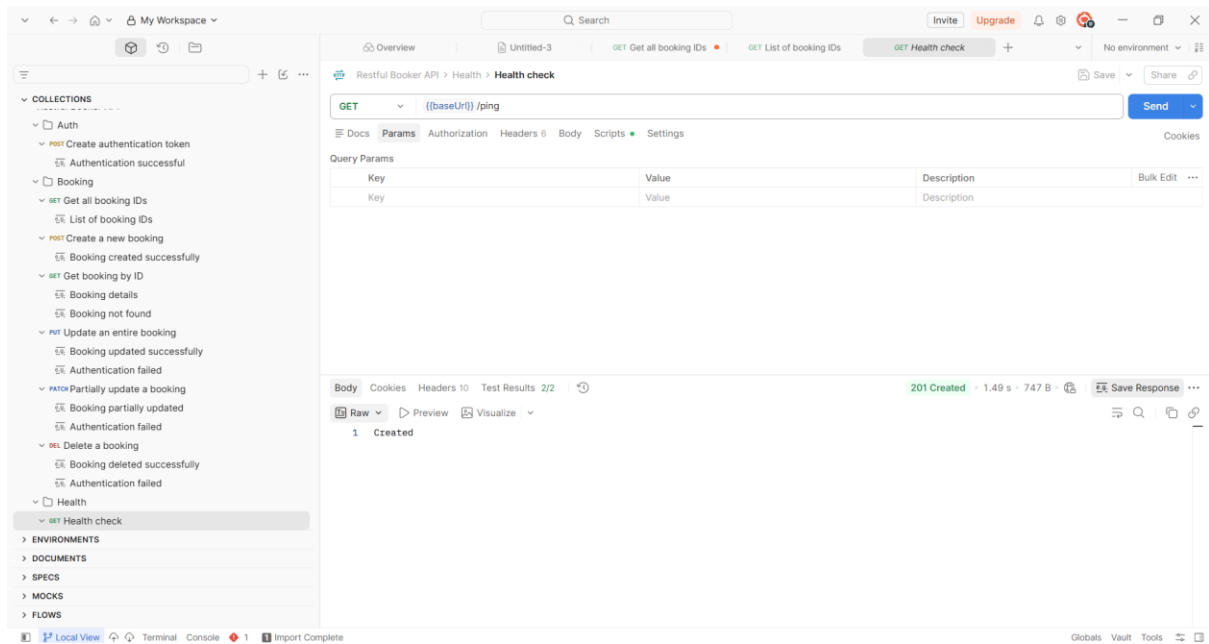
```
1 {
2   "bookingid": 2424,
3   "booking": {
4     "firstname": "Jim",
5     "lastname": "Brown",
6     "totalprice": 111,
7     "depositpaid": true,
8     "bookingdates": {
9       "checkin": "2026-08-10",
10      "checkout": "2026-08-15"
11    },
12    "additionalneeds": "Breakfast"
13  }
14 }
```

Get booking details

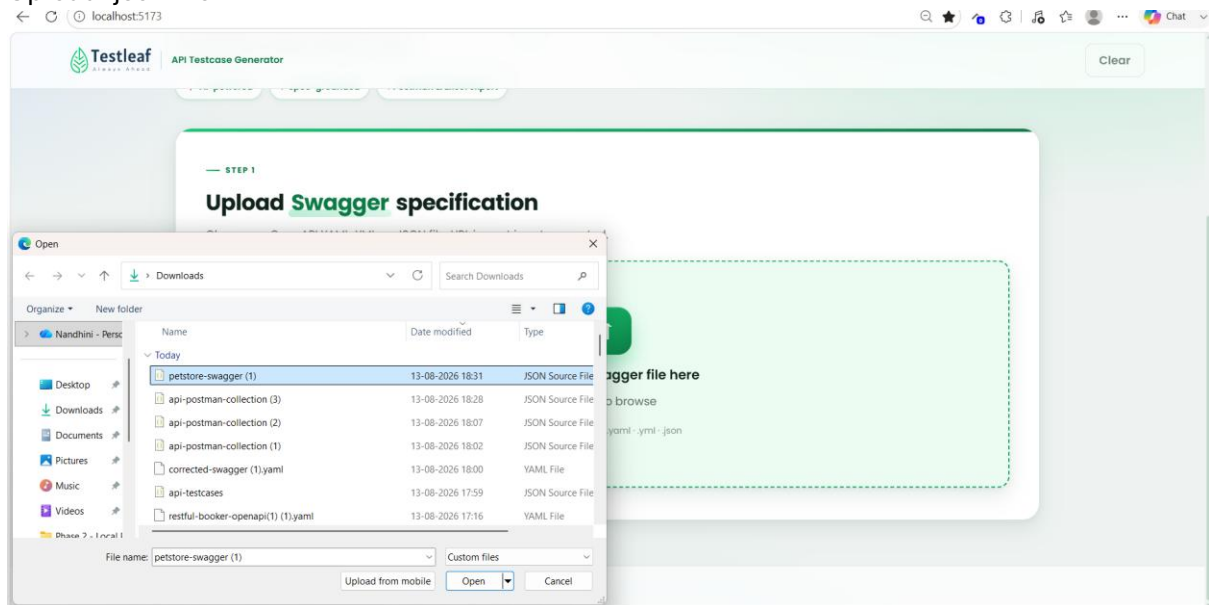
The screenshot shows the Postman interface with the "Booking details" endpoint selected. The request is a GET to `{{baseUri}}/booking/3400`. The response is a 200 OK status with a JSON body:

```
1 {
2   "firstname": "Jim",
3   "lastname": "Brown",
4   "totalprice": 111,
5   "depositpaid": true,
6   "bookingdates": {
7     "checkin": "2026-08-10",
8     "checkout": "2026-08-15"
9   },
10  "additionalneeds": "Breakfast"
11 }
```

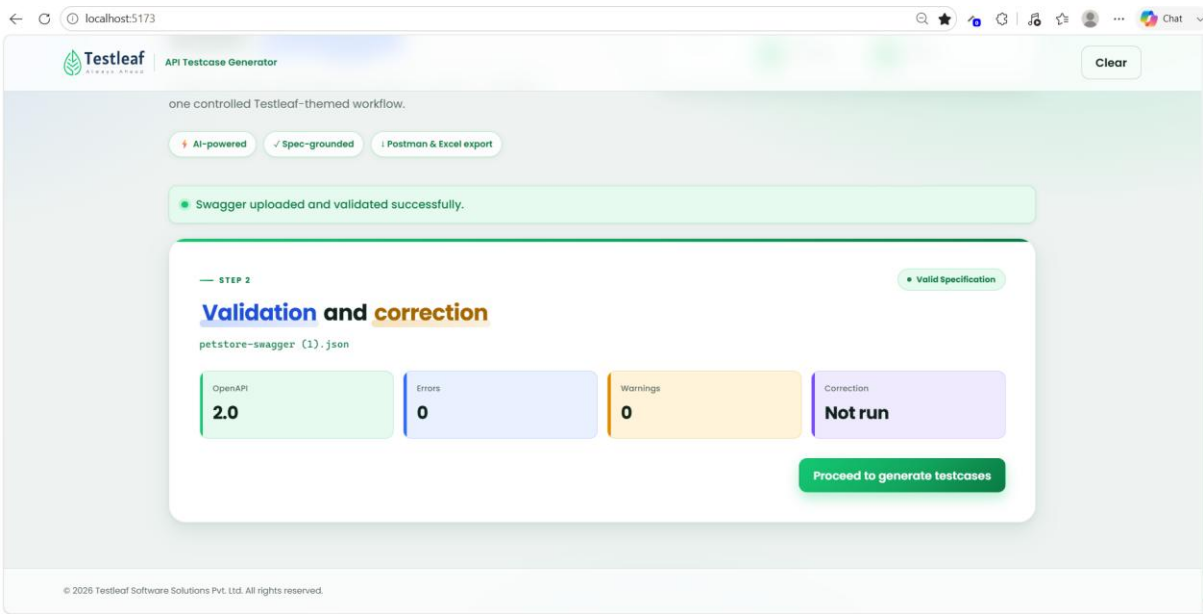
Get Health check



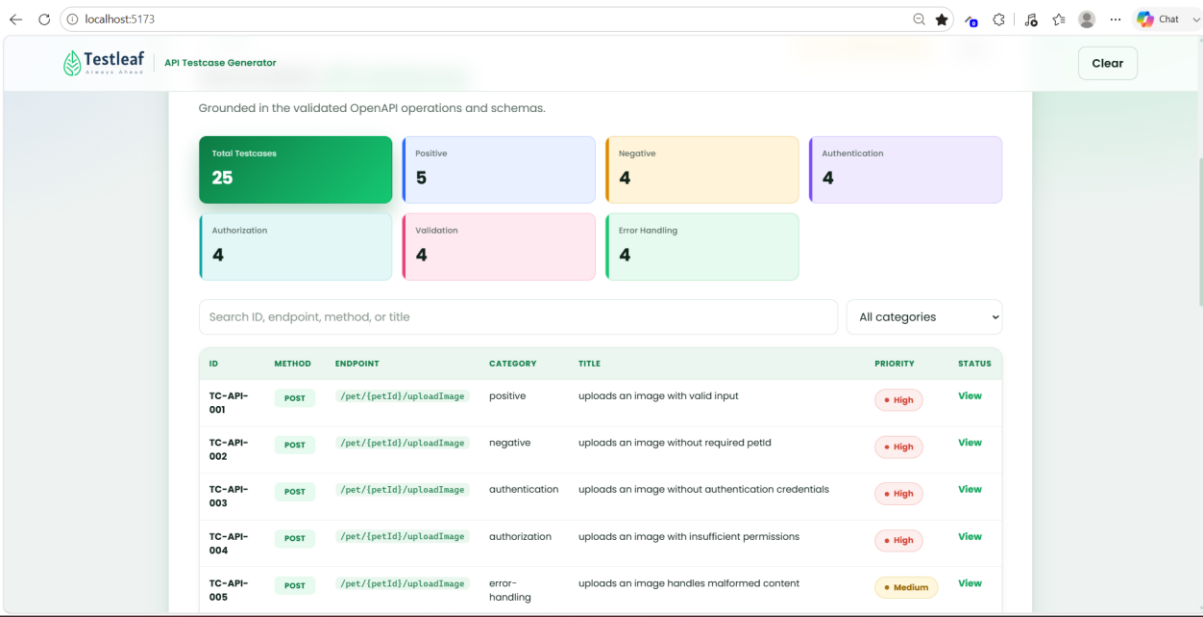
Upload .json file



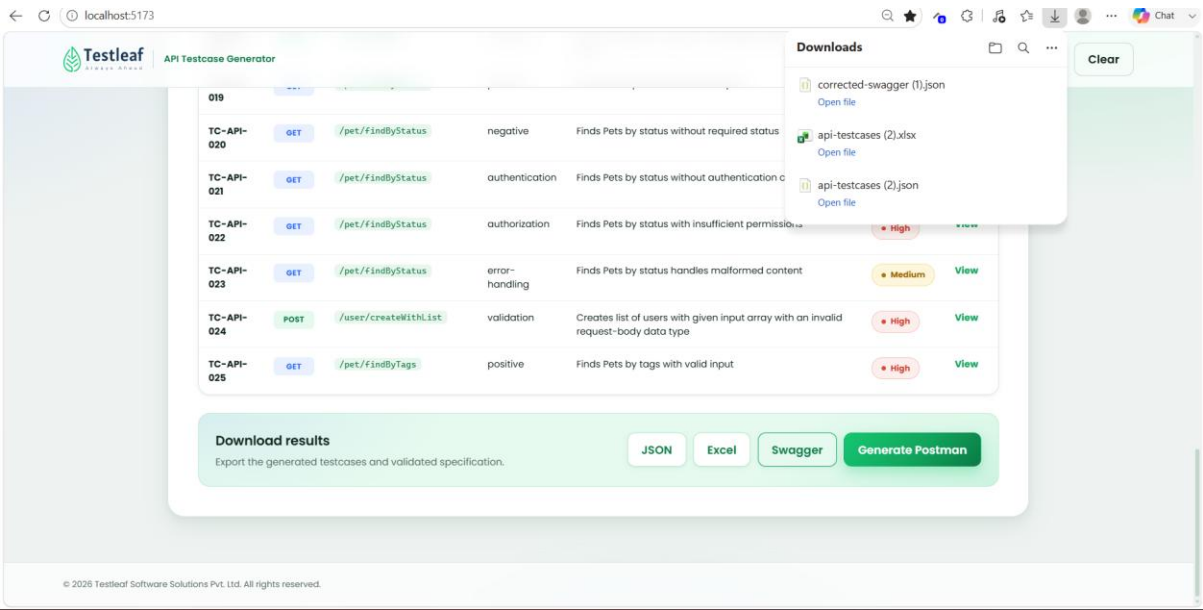
Processing is done with no errors



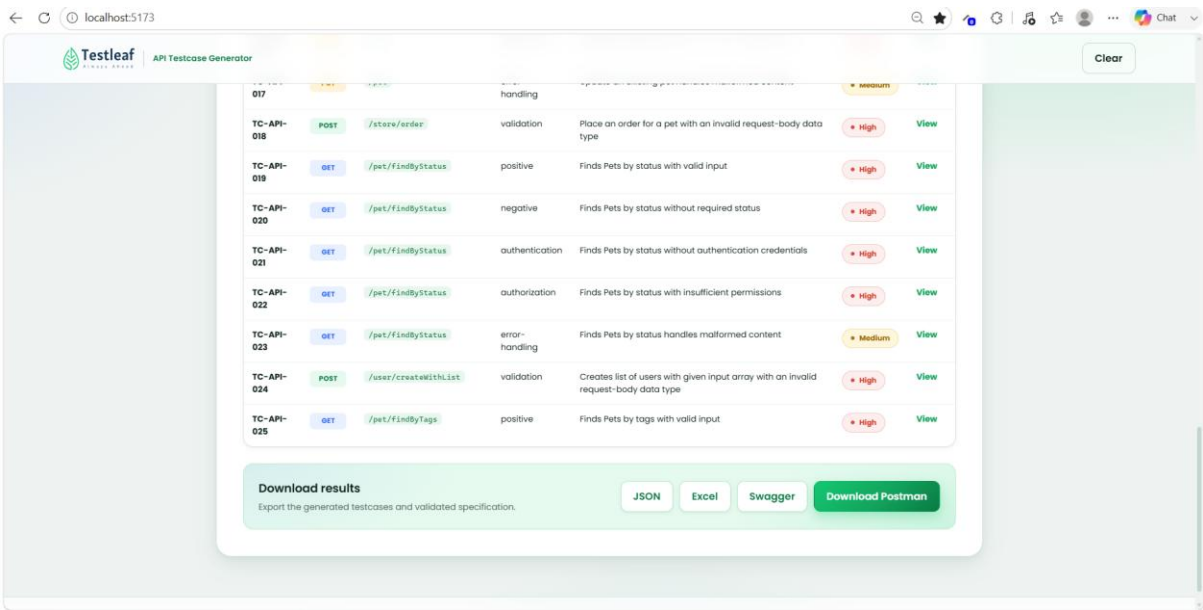
Click on Proceed to generate testcases



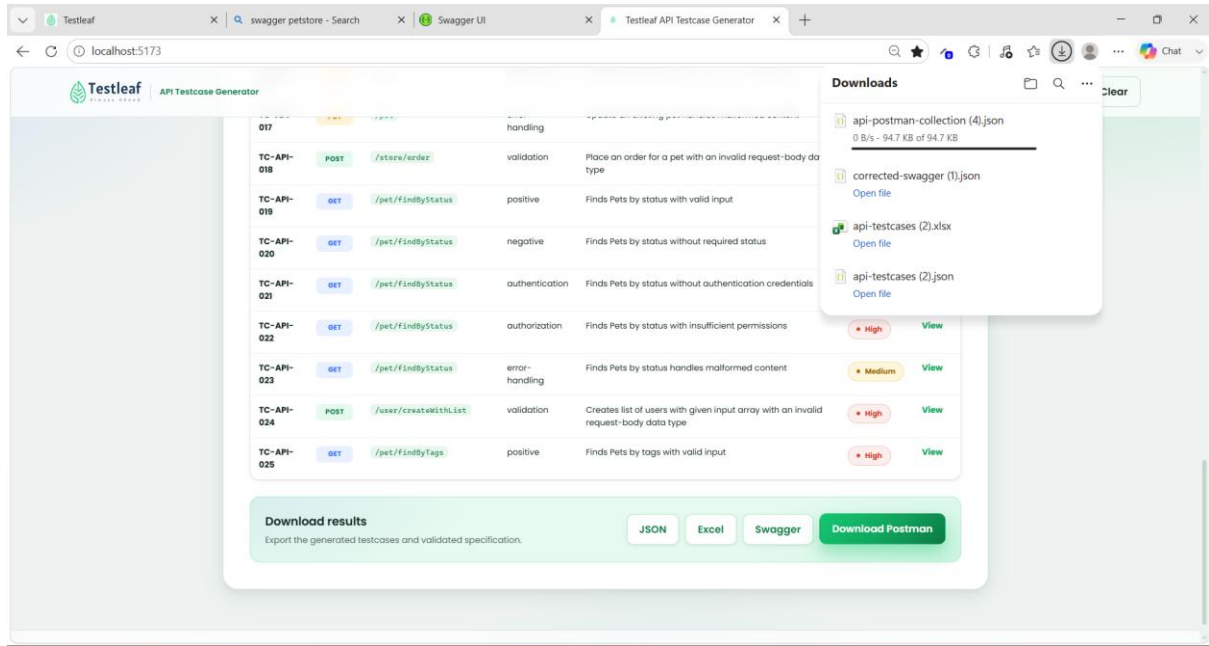
Export the generated test cases through Json/Excel/Swagger



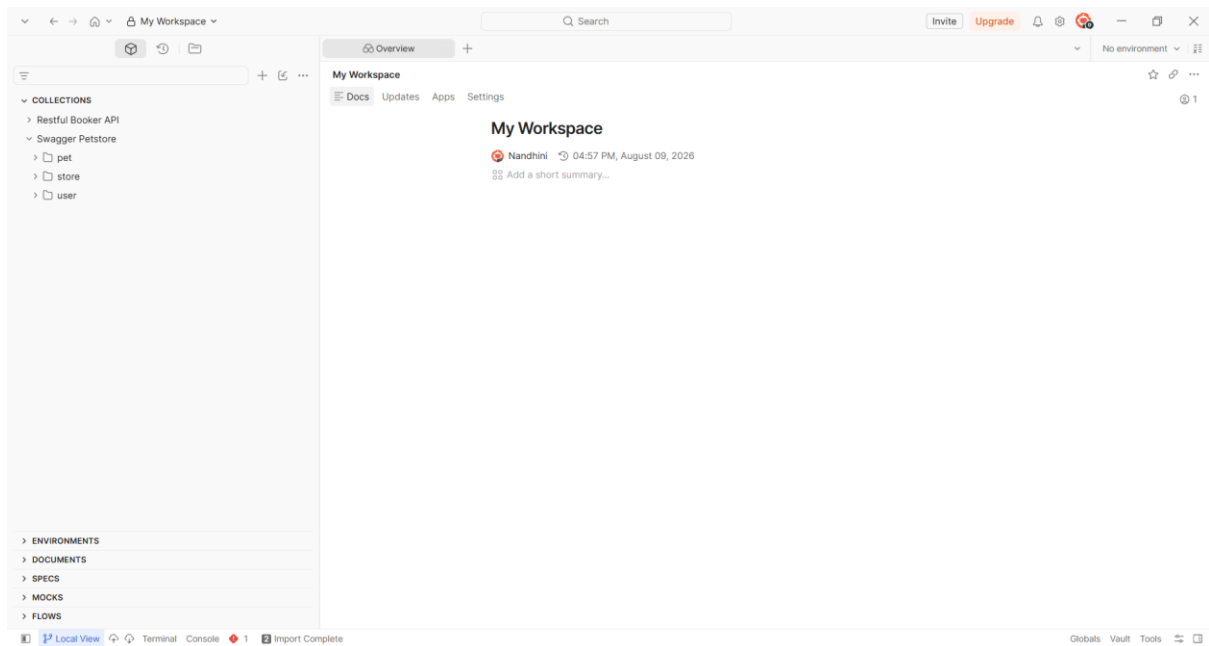
Generate Postman Collection is clicked, it will ask for download postman



Click Download Postman



Open the downloaded Postman file in Postman



Upload image

The screenshot shows the Swagger Petstore API interface. The left sidebar lists collections: Restful Booker API, Swagger Petstore, pet, and user. The 'pet' collection is expanded, showing endpoints like 'POST uploads an image', 'POST Add a new pet to the store', 'PUT Update an existing pet', 'GET Finds Pets by status', 'GET Finds Pets by tags', 'GET Find pet by ID', 'POST Updates a pet in the store with form data', 'DELETE Deletes a pet', 'store', and 'user'. The 'POST uploads an image' endpoint is selected. The main panel shows the endpoint details: POST {{baseUri}}/pet/7342/uploadImage. The 'Headers' tab is active, showing a table with headers: Content-Type (multipart/form-data) and Accept (application/json). The 'Body' tab is also active, showing a JSON response: { "code": 200, "type": "unknown", "message": "additionalMetadata: string\nFile uploaded to ./Hospital.png, 17940 bytes" }. The status is 200 OK, 325 ms, 440 B.

Key	Value	Description	Bulk Edit	Presets
Content-Type	multipart/form-data			
Accept	application/json	Description		

```
{
  "code": 200,
  "type": "unknown",
  "message": "additionalMetadata: string\nFile uploaded to ./Hospital.png, 17940 bytes"
}
```

Add a new pet to the store

The screenshot shows the Swagger Petstore API interface. The left sidebar lists collections: Restful Booker API, Swagger Petstore, pet, and user. The 'pet' collection is expanded, showing endpoints like 'POST uploads an image', 'POST Add a new pet to the store', 'PUT Update an existing pet', 'GET Finds Pets by status', 'GET Finds Pets by tags', 'GET Find pet by ID', 'POST Updates a pet in the store with form data', 'DELETE Deletes a pet', 'store', and 'user'. The 'POST Add a new pet to the store' endpoint is selected. The main panel shows the endpoint details: POST {{baseUri}}/pet. The 'Params' tab is active, showing a table with parameters: Key (Key) and Value (Value). The 'Body' tab is also active, showing a JSON response: { "id": 3995, "name": "string", "name": "doggie", "photoUrls": ["string", "string"], "tags": [{ "id": 8240, "name": "string" }] }. The status is 200 OK, 228 ms, 505 B.

Key	Value	Description	Bulk Edit
Key	Value	Description	

```
{
  "id": 3995,
  "name": "string",
  "name": "doggie",
  "photoUrls": [
    "string",
    "string"
  ],
  "tags": [
    {
      "id": 8240,
      "name": "string"
    }
  ]
}
```

Update an existing pet

The image shows the Swagger UI interface for the Swagger Petstore API. The left sidebar displays the API structure, with the 'Update an existing pet' endpoint selected under the 'pet' collection. The main panel shows the endpoint details for the PUT method, including the URL, parameters, and a response body. The response body is a JSON object representing a pet.

Endpoint: PUT `{{baseUrl}}/pet`

Query Params:

Key	Value	Description	Bulk Edit
Key	Value	Description	

Response Body (JSON):

```
1 {
2   "id": 2926,
3   "category": {
4     "id": 7257,
5     "name": "string"
6   },
7   "name": "doggie",
8   "photoUrls": [
9     "string",
10    "string"
11  ],
12  "tags": [
13    {
14      "id": 7339,
```

Find pet by status

The image shows the Swagger UI interface for the Swagger Petstore API. The left sidebar displays the API structure, with the 'Find pet by status' endpoint selected under the 'pet' collection. The main panel shows the endpoint details for the GET method, including the URL, parameters, and a response body. The response body is a JSON array of pet objects.

Endpoint: GET `{{baseUrl}}/pet/findByStatus?status=available&status=available`

Query Params:

Key	Value	Description	Bulk Edit
☒ status	available	(Required) Status values that need to be considered for fi	
☒ status	available	(Required) Status values that need to be considered for fi	
Key	Value	Description	

Response Body (JSON):

```
1 [
2   {
3     "id": 9223372036854775249,
4     "category": {
5       "id": 0,
6       "name": "string"
7     },
8     "name": "doggie",
9     "photoUrls": [
10      "string"
11    ],
12    "tags": [
13      {
14        "id": 0,
```

Find pet by tags

Swagger Petstore > pet > **Find Pets by tags**

GET {{baseUri}}/pet/findByTags?tags=string&tags=string

Query Params

Key	Value	Description
tags	string	(Required) Tags to filter by
tags	string	(Required) Tags to filter by
Key	Value	Description

Body

```
{
  "category": {
    "id": 0,
    "name": "string"
  },
  "name": "doggie",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ]
}
```

200 OK • 2.25 s • 69.5 KB

Find Pet by ID

Swagger Petstore > pet > **Find pet by ID**

GET {{baseUri}}/pet/:petId

Path Variables

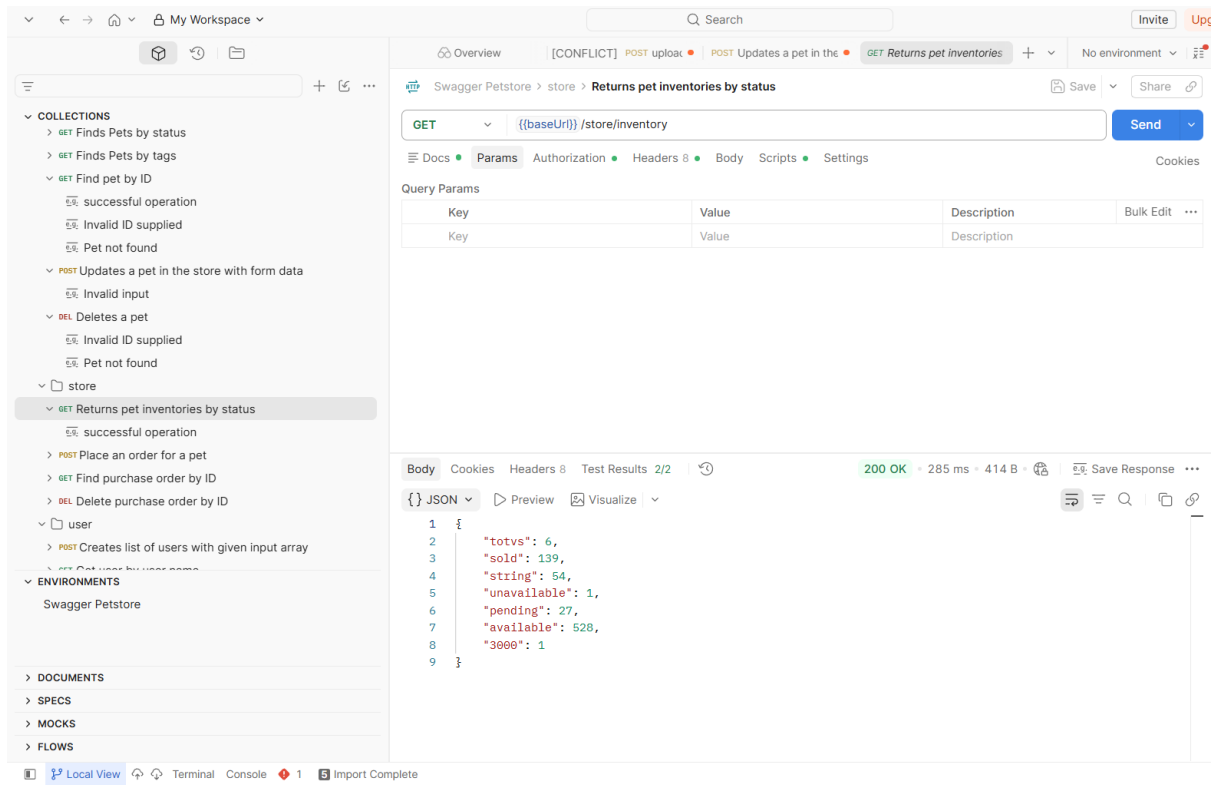
Key	Value	Description
petId	1	(Required) ID of pet to return

Body

```
{
  "id": 1,
  "category": {
    "id": 1,
    "name": "cat"
  },
  "name": "dog",
  "photoUrls": [],
  "tags": [],
  "status": "sold"
}
```

200 OK • 251 ms • 421 B

Returns pet inventories by status



Swagger Petstore > store > Returns pet inventories by status

GET {{baseUri}}/store/inventory

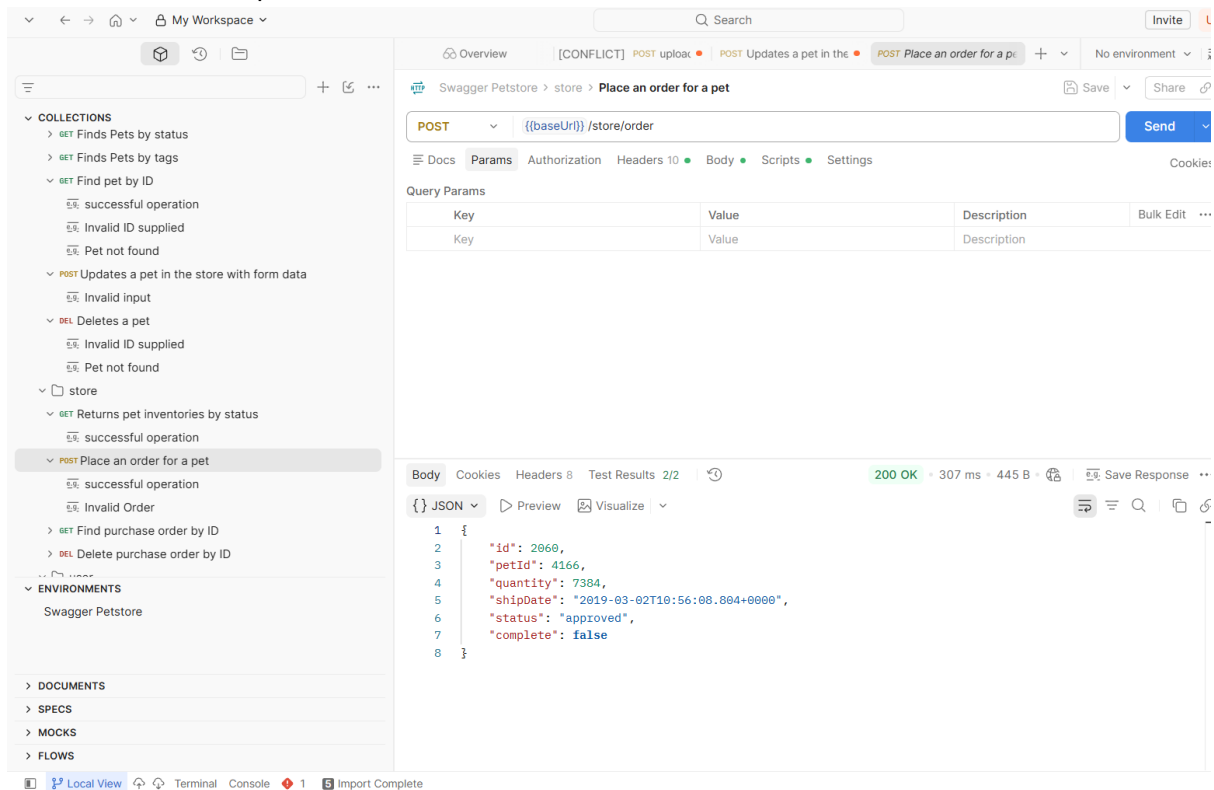
Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 8 Test Results 2/2 200 OK • 285 ms • 414 B Save Response

```
{
  "totvs": 6,
  "sold": 139,
  "string": 54,
  "unavailable": 1,
  "pending": 27,
  "available": 528,
  "3000": 1
}
```

Place an order for a pet



Swagger Petstore > store > Place an order for a pet

POST {{baseUri}}/store/order

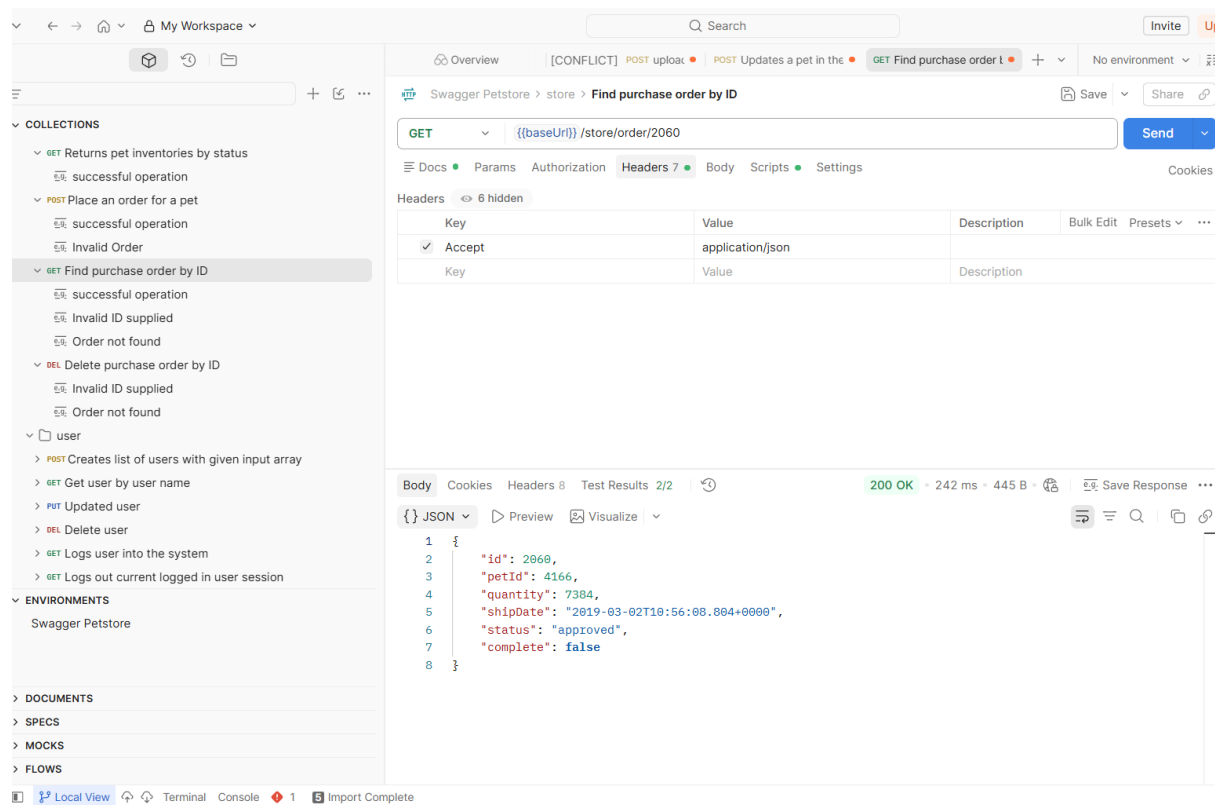
Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 8 Test Results 2/2 200 OK • 307 ms • 445 B Save Response

```
{
  "id": 2060,
  "petId": 4166,
  "quantity": 7384,
  "shipDate": "2019-03-02T19:56:08.804+0000",
  "status": "approved",
  "complete": false
}
```

Find purchase by order ID



Gave the prompt as Modify the **existing API Testcase Generator application** shown in the current UI.

Objective

Add a **Swagger/OpenAPI URL input option** so users can enter a Swagger specification URL instead of uploading a Swagger file.

Example URL:

<https://petstore.swagger.io/>

UI Changes

In the existing **Step 1 – Upload Swagger specification** section:

1. Keep the existing file-upload functionality unchanged.
2. Add a clearly visible **"OR"** separator below the file upload area.
3. Add a textbox/input field with:
 - Label: **Swagger URL**
 - Placeholder: **Enter Swagger/OpenAPI specification URL**
 - Example value: <https://petstore.swagger.io/>
1. The UI should remain consistent with the existing Testleaf design:

- Green theme
 - Existing fonts, spacing, cards, borders and buttons
 - Responsive layout
2. Add basic URL validation and show a user-friendly validation message for an invalid/empty URL.

URL Processing

When the user enters:

`https://petstore.swagger.io/`

the application should retrieve the Swagger/OpenAPI specification from that URL.

Important:

- Do NOT assume that the URL itself is always the raw JSON/YAML specification.
- Handle Swagger UI URLs where possible.
- Detect the actual OpenAPI/Swagger specification URL from the Swagger UI configuration.
- Support OpenAPI JSON/YAML and Swagger 2.0 specifications.
- If the specification cannot be retrieved, display a clear error message.

Backend

If the existing application has a Node.js/Express backend, create an API endpoint such as:

POST `/api/swagger/import-url`

Request:

```
{  
  "url": "https://petstore.swagger.io/"  
}
```

The backend should:

1. Validate the URL.
2. Fetch the Swagger/OpenAPI specification.
3. Parse JSON/YAML as appropriate.
4. Validate that it is a valid Swagger/OpenAPI specification.
5. Return the parsed specification to the frontend.
6. Reuse the application's existing test-case-generation logic rather than creating a separate generation implementation.

Test Case Generation

After successfully retrieving the Swagger specification:

1. Extract all available API endpoints.
2. Extract HTTP methods.
3. Extract path parameters.
4. Extract query parameters.
5. Extract request headers.
6. Extract request body/schema.
7. Extract response codes and response schemas.
8. Generate test cases using the application's existing test-case-generation workflow.

Generate positive, negative and boundary test scenarios where applicable.

For example, for the Petstore API, generate test cases for endpoints such as:

- GET
- POST
- PUT
- DELETE

Do not hard-code Petstore endpoints. The implementation must work with any valid Swagger/OpenAPI specification.

Loading and Error Handling

While the Swagger URL is being processed:

- Disable the Generate button.
- Display a loading indicator such as **"Fetching Swagger specification..."**
- Prevent duplicate requests.

Handle:

- Invalid URL
- URL not reachable
- Timeout
- Invalid Swagger/OpenAPI specification
- Unsupported specification format
- CORS/network errors
- Empty specification
- Backend errors

Display meaningful messages instead of raw stack traces.

Important Existing-App Requirement

Do NOT rebuild the application from scratch.

Inspect the existing frontend, backend, API routes, services, components and test-case-generation logic first.

Reuse existing:

- Components
- Styling
- API services
- Swagger parsing logic
- Test-case generation logic
- State management
- Error handling
- Export functionality

Only add or modify the minimum required files.

Expected User Flow

Upload Swagger file

OR

Enter Swagger URL

↓

Click "Generate Test Cases"

↓

Fetch Swagger/OpenAPI specification

↓

Validate specification

↓

Extract API endpoints

↓

Generate test cases using existing generator

↓

Display generated test cases

↓

Allow existing Postman / Excel export functionality

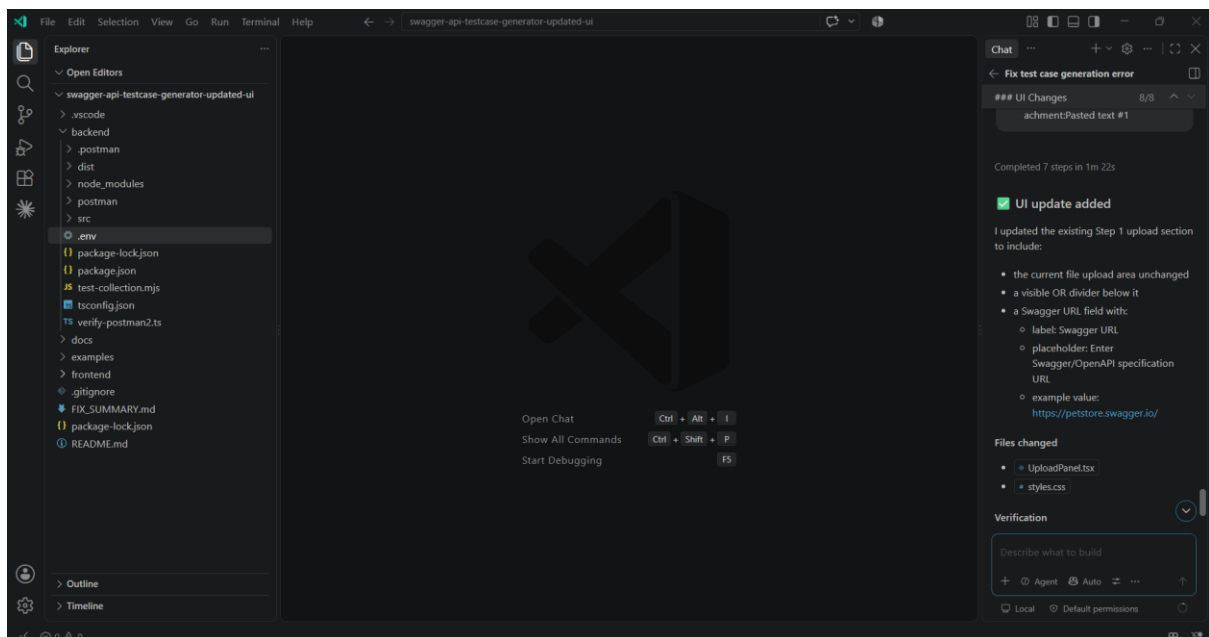
Acceptance Criteria

1. Existing file upload continues to work exactly as before.
2. User can enter <https://petstore.swagger.io/>.
3. Clicking **Generate Test Cases** retrieves the Swagger specification.

4. The specification is validated before generation.
5. Test cases are generated from the retrieved specification.
6. No Petstore-specific hardcoding is introduced.
7. Invalid URLs produce user-friendly errors.
8. Loading state is displayed while fetching/generating.
9. Existing generated-test-case UI remains unchanged unless necessary.
10. Existing Postman and Excel export functionality continues to work.
11. Do not introduce unnecessary dependencies.
12. Keep the implementation clean, modular and production-ready.

Before making changes, inspect the existing project structure and identify the correct frontend and backend files to modify.

Implemented textbox for entering <https://petstore.swagger.io/> based on the URL it should generate test cases



Implemented the same through Vibe coding

The screenshot shows the 'STEP 1: Upload Swagger specification' interface. It features a large dashed green box with an upward arrow icon and the text 'Drop your Swagger file here or click to browse'. Below this, a 'Swagger URL' input field contains 'https://petstore.swagger.io/'. To the right of the input field is a green 'Generate Test Cases' button. Above the input field, there is a small 'OR' separator. The interface also includes a 'Clear' button in the top right corner and a 'Chat' icon in the top right corner of the browser window.

Testleaf API Testcase Generator

STEP 1

Upload Swagger specification

Choose an OpenAPI YAML, YML, or JSON file, or paste a Swagger URL.

Drop your Swagger file here
or click to browse

Maximum 5 MB : yaml : yml : json

OR

Swagger URL

Generate Test Cases

Example: <https://petstore.swagger.io/>

Click Generate test cases and it is processed without any errors

The screenshot shows the 'STEP 2: Validation and correction' interface. It features a progress bar at the top with four steps: 'Upload', 'Generate', 'Export', and 'Correct'. The 'Generate' step is currently active. Below the progress bar, there is a green notification bar that says 'Swagger URL processed and validated successfully.' The main section is titled 'Validation and correction' and shows the 'swagger.json' file. It displays four metrics: 'OpenAPI 2.0', 'Errors 0', 'Warnings 0', and 'Correction Not run'. A green 'Proceed to generate testcases' button is located at the bottom right of the main section. The interface also includes a 'Clear' button in the top right corner and a 'Chat' icon in the top right corner of the browser window.

Testleaf API Testcase Generator

from Swagger

Upload, validate, correct, generate, review, and export — through one controlled Testleaf-themed workflow.

AI-powered Spec-grounded Postman & Excel export

Swagger URL processed and validated successfully.

STEP 2

Validation and correction

swagger.json

OpenAPI 2.0

Errors 0

Warnings 0

Correction Not run

Proceed to generate testcases

© 2025 Testleaf Software Solutions Pvt. Ltd. All rights reserved.

Click Proceed to generate testcases

The screenshot displays the Testleaf API Testcase Generator interface. At the top, the browser address bar shows 'localhost:5173'. The Testleaf logo and 'API Testcase Generator' are in the top left. A 'Clear' button is in the top right. The main heading is 'Generated API testcases', with a subtext 'Grounded in the validated OpenAPI operations and schemas.' Below this, a summary section shows a total of 25 testcases, broken down by category: Positive (5), Negative (4), Authentication (4), Authorization (4), and Validation (4). A search bar and a category filter dropdown are present. The main table lists 5 testcases, each with an ID, method (POST), endpoint, category, title, priority, and status.

ID	METHOD	ENDPOINT	CATEGORY	TITLE	PRIORITY	STATUS
TC-API-001	POST	/pet/{petId}/uploadImage	positive	uploads an image with valid input	High	View
TC-API-002	POST	/pet/{petId}/uploadImage	negative	uploads an image without required petid	High	View
TC-API-003	POST	/pet/{petId}/uploadImage	authentication	uploads an image without authentication credentials	High	View
TC-API-004	POST	/pet/{petId}/uploadImage	authorization	uploads an image with insufficient permissions	High	View
TC-API-005	POST	/pet/{petId}/uploadImage	error-handling	uploads an image handles malformed content	Medium	View

[Download JSON/EXCEL/SWAGGER](#)

←

↻

localhost:5173

Testleaf

API Testcase Generator

TC-API-018	POST	/store/order	validation	Place an order for a pet with an invalid request-body type		
TC-API-019	GET	/pet/findByStatus	positive	Finds Pets by status with valid input.		
TC-API-020	GET	/pet/findByStatus	negative	Finds Pets by status without required status		
TC-API-021	GET	/pet/findByStatus	authentication	Finds Pets by status without authentication creden,		
TC-API-022	GET	/pet/findByStatus	authorization	Finds Pets by status with insufficient permissions	High	View
TC-API-023	GET	/pet/findByStatus	error-handling	Finds Pets by status handles malformed content	Medium	View
TC-API-024	POST	/user/createWithList	validation	Creates list of users with given input array with an invalid request-body data type	High	View
TC-API-025	GET	/pet/findByTags	positive	Finds Pets by tags with valid input	High	View

Downloads

corrected-swagger (3).json
Open file

api-testcases (4).xlsx
Open file

api-testcases (5).json
Open file

Clear

Download results

Export the generated testcases and validated specification.

JSON

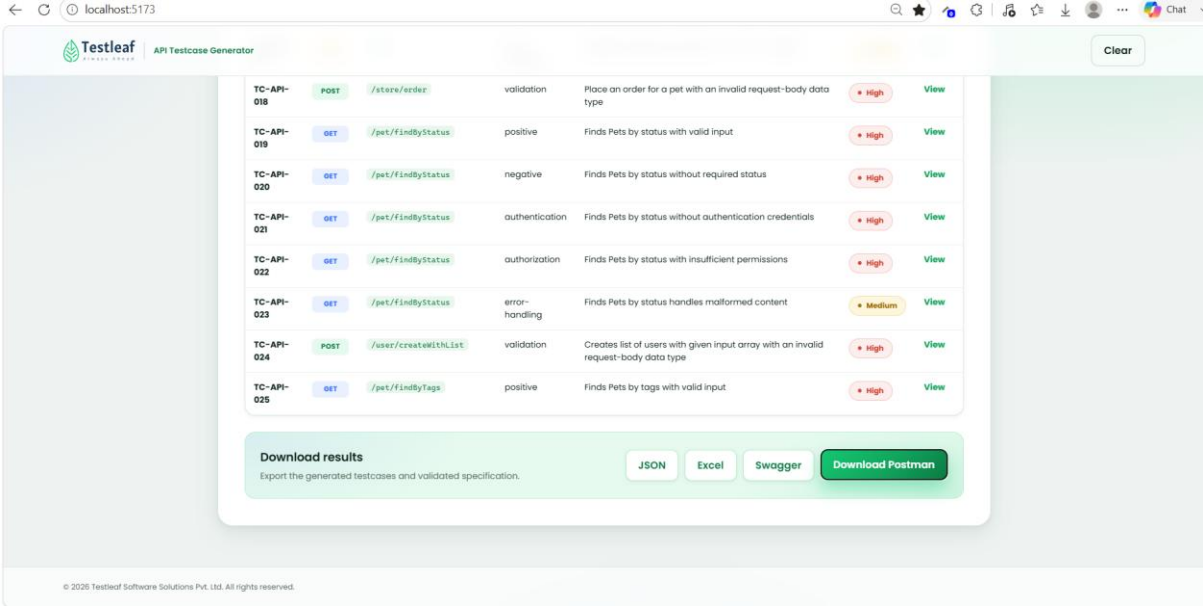
Excel

Swagger

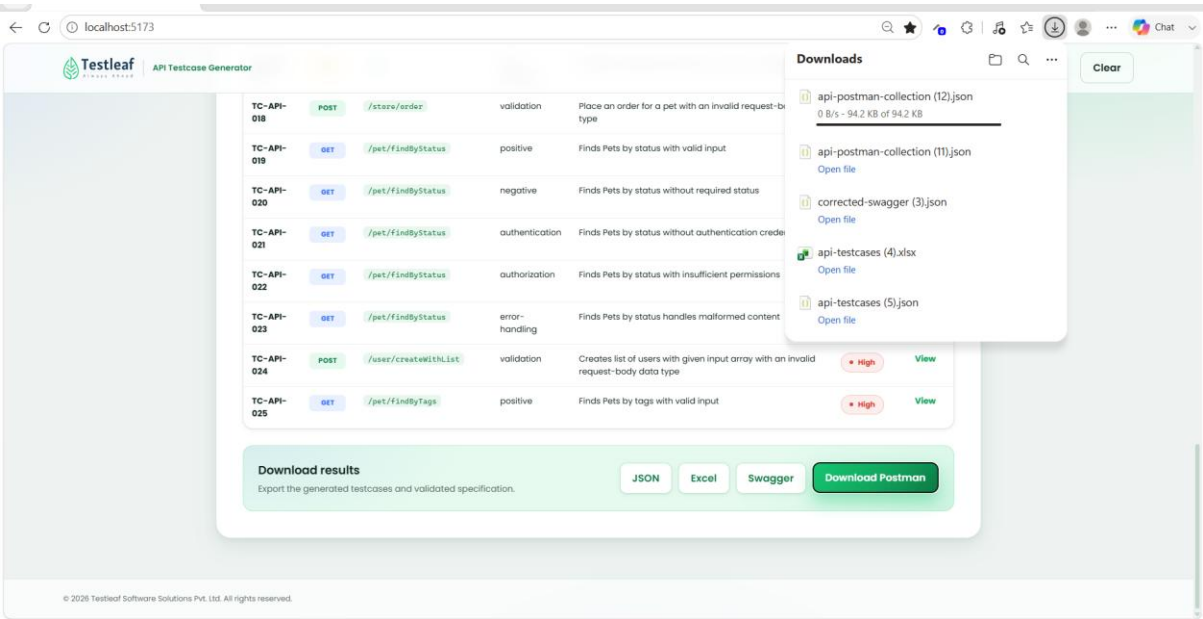
Generate Postman

© 2026 Testleaf Software Solutions Pvt. Ltd. All rights reserved.

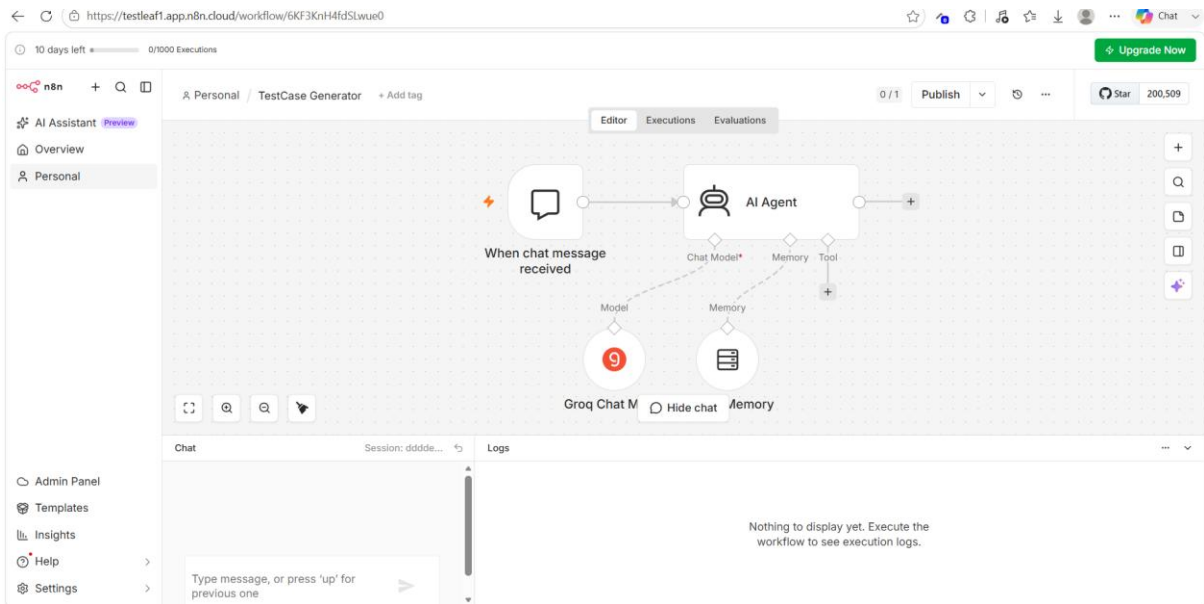
Click Generate Postman



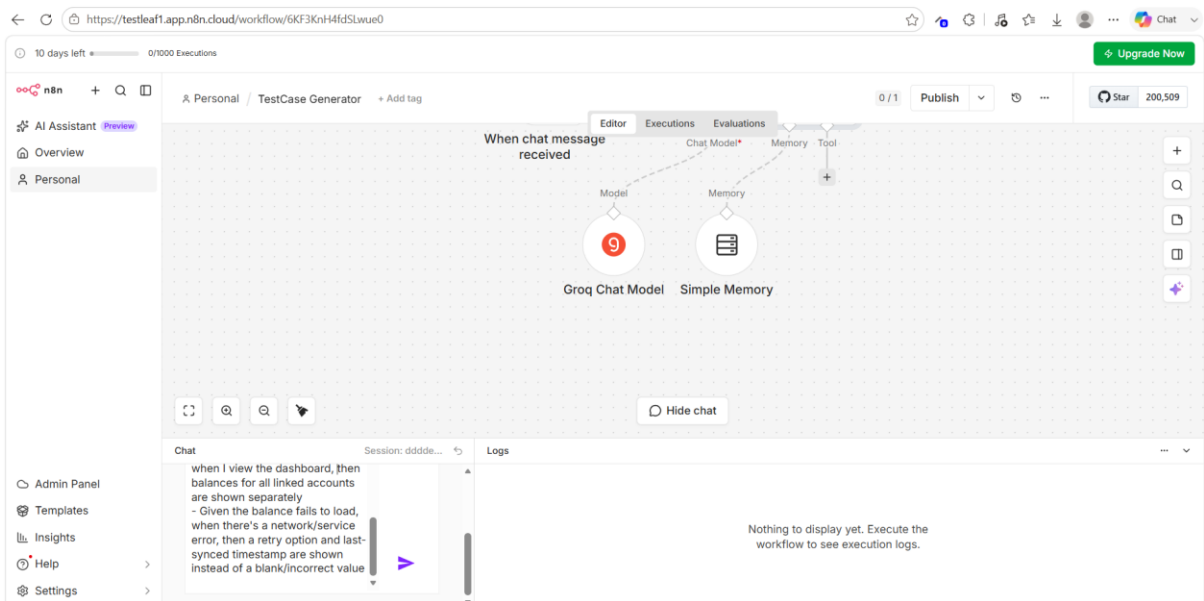
Download the Postman



2. N8N



Giving the user stories in chat



Test cases are generated

The screenshot displays the n8n workflow editor for a workflow named 'Test Case Generator'. The workflow is triggered by 'When chat message received' and involves an 'AI Agent' node connected to 'Groq Chat Model' and 'Simple Memory' nodes. The execution log shows a successful run with the following details:

- Chat:** Session: be90a...
displayed Expected Result: The account balance is accurately displayed as zero without any errors
- Logs:** Success in ... 1,362 T...
AI Agent
Simple Me...
Groq Chat ...
Simple Me...
- INPUT:** System: Strict Test Case Generator (v1) - Instruction: You are a Test Case Generation Engine... Your task is to generate high-quality, detailed test cases ONLY when the user explicitly asks for test cases related to a valid software/system functionality, provided either as text (user story, BRD, requirement) or as an image (UI screenshot, wireframe, mockup, flow diagram)... You must cover: - Positive scenarios -
- OUTPUT:** Expected Result: The current balance and available balance are accurately displayed
Test Case ID: TC_AB_002
Title: Verify account balance display for multiple accounts
Preconditions: User is logged in with multiple active accounts
Steps:

For validating simple memory giving the Generate 3 more negative test cases, 8 test cases generated

The screenshot displays the n8n workflow editor for the same 'Test Case Generator' workflow. The execution log shows a successful run with the following details:

- Chat:** Session: a166f...
message is displayed indicating the session has expired and the user is prompted to log in again
- Logs:** Success in ... 1,683 To...
When chat mess...
AI Agent
Simple Me...
Groq Chat ...
Simple Me...
- INPUT:** action saveContext
input
input Generate 3 more negative test cases
output
output Test Case ID: TC_CB_006 Title:
- OUTPUT:** Test Case ID: TC_CB_008
Title: Verify account balance display with expired session
Preconditions: User's session has expired due to inactivity
Steps:
Allow the session to expire due to inactivity
Attempt to navigate to the "Accounts" tab
Verify an error message is displayed indicating the session